



IGNITION IO

Crucible Linux User Guide

The **Crucible** Workbench and the **Ignny** CLI

Set up a chip-design toolchain, run digital and analog experiments,
and reproduce the exact same environment on any machine.

CONTENTS

What's in this guide

Read sections 1 to 3 once. After that, jump to the flow you need. Every command in this guide is copy-paste ready for a Linux shell.

01	Before You Begin	What the product is, in plain words
02	Setup in Five Minutes	From a blank Linux machine to a working lab
03	Command Reference	Every igny command, with examples
04	Digital Flow	Verilog: compile, simulate, view waveforms
05	Synthesis Flow	RTL to gate netlist with Yosys
06	Analog Flow	Ngspice decks and design-equation checks
07	Sharing and Reproducing Work	Lock, sync, hand off to a teammate
08	Linux-Specific Notes	Distros, WSL2, and the one missing dependency
09	Troubleshooting	Common messages and what to do

SECTION 1 · GETTING ORIENTED

Before You Begin

What Crucible does, and the three words you need to understand it.

THE PROBLEM THIS SOLVES

Chip design needs a stack of tools - simulator, waveform viewer, synthesis engine, **SPICE** engine, cell library. Normally you install each by hand, match versions by **hand**, and fight **\$PATH** conflicts by hand.

Crucible is a managed workbench: it installs, versions and runs those tools for you. Nothing needs **sudo**. **Ignpy** is the command you type - the CLI to Crucible.

THREE WORDS TO LEARN

WORD	THINK OF IT AS
Environment	A named toolbox
Workspace	Your project folder
Tool	The actual program

The one rule to remember

Always run ignpy commands from the workspace root - the folder that contains crucible.toml.

Required on your machine

A 64-bit Linux distro (Ubuntu, Debian, Fedora, Arch, or WSL2) · Ignpy CLI on your \$PATH · build-essential (or your distro's equivalent - section 8) · internet for the first install · a bash or zsh terminal. Not required: sudo, manual \$PATH edits, a licence server.

TOOLS USED IN THIS GUIDE

INSTALL NAME	WHAT IT IS	USED FOR
iverilog	Icarus Verilog	Compiles Verilog + testbench into a runnable sim.
gtkwave	Waveform viewer	Opens the .vcd a simulation produces.
yosys	Synthesis engine	Turns behavioural RTL into a gate-level netlist.
ngspice	Analog simulator	Runs SPICE decks - DC, transient, transistor-level.
matplotlib	Python plotting	Draws design-verification plots.

SECTION 2 · FROM ZERO

Setup in Five Minutes

Do this once. Every experiment afterwards reuses it.

1 Start Crucible

Run this once on a new machine - no sudo needed.

```
$ igny init
```

2 Create an environment

Pick any name you like. This is the toolbox your project will use.

```
$ igny env create verilog-lab
```

3 Create a workspace and go into it

--path is the folder to create. --env attaches it to the environment above.

```
$ igny workspace create --path ./my-lab --env verilog-lab  
$ cd ./my-lab
```

INSTALL THE TOOLS YOU NEED, THEN LOCK

```
$ igny tool install iverilog  
$ igny tool install gtkwave  
$ igny tool install yosys      # only if you will synthesize  
$ igny tool install ngspice   # only if you will run analog  
$ igny workspace lock         # freezes exact versions
```

SECTION 2 · FROM ZERO

What You Should See

```
you@linux:~$ igny env create verilog-lab
Environment 'verilog-lab' created (stack=default)
you@linux:~$ igny tool install iverilog
✓ Installed iverilog
Source: artifact
Path: /home/you/Crucible/tools/iverilog/12.0
Bound to env: verilog-lab
```

The install prints the version and install path under your **home directory** - not **/usr** or **/opt**. Nothing is added to your system **\$PATH**. That is intentional: it is why **igny run** exists.

HOW TO ORGANISE YOUR FILES

```
my-lab/
├─ crucible.toml ← the workspace marker, do not delete
├─ 01_dff/
├─ 02_counter/
└─ 03_amplifier/
```

Check it worked

A tick and a Bound to env line means the tool is ready. Nothing was added to your system PATH - that is by design.

SECTION 3 · REFERENCE

Command Reference

Every command you will use, grouped by what it is for.

SETTING UP AND MANAGING TOOLS

COMMAND	WHAT IT DOES
<code>ign y init</code>	Prepares Crucible on this machine. Run once.
<code>ign y env create <name></code>	Creates a named environment (a toolbox).
<code>ign y workspace create --path <dir> --env <name></code>	Creates a project folder, attaches it.
<code>ign y tool install <tool></code>	Downloads a catalog tool once, binds it.
<code>ign y workspace lock</code>	Freezes current tool versions.
<code>ign y workspace sync</code>	Installs exact versions from the lock.

RUNNING TOOLS

COMMAND	WHAT IT DOES
<code>ign y run <tool> <args></code>	Runs a tool. Arguments pass through verbatim.
<code>ign y run script <file.py></code>	Runs a Python script inside the environment.

How ign y run works

Think of ign y run as a prefix. If a tool's docs say `yosys -s script.tcl`, you type `ign y run yosys -s script.tcl` - the arguments pass straight through, exactly like calling the tool directly. One tool can provide several commands: installing iverilog also gives you `vvp`.

SECTION 4 · VERILOG SIMULATION

Digital Flow

Write a design, test it, and look at the waveform. This is the flow you will use most.

THE THREE COMMANDS

STEP	COMMAND	WHAT HAPPENS
Compile	<code>iverilog -o sim.vvp -s tb design.v tb.v</code>	Builds a runnable sim file.
Simulate	<code>vvp sim.vvp</code>	Runs it, writes a .vcd.
View	<code>gtkwave tb.vcd</code>	Opens the waveform viewer.

RUN IT

```
$ cd 01_dff
$ igny run iverilog -o dff_sim.vvp -s dff_tb dff.v dff_tb.v
$ igny run vvp dff_sim.vvp
$ igny run gtwave dff_tb.vcd
```

How to tell pass from fail

'... test passed.' = PASS. 'ERROR: ...' = FAIL, the testbench stops at the first mismatch. '\$finish called at ...' alone is not a pass - look for the pass line above it.

On a headless machine, **GTKWave** needs a display. Forward **X11** (`ssh -X`), or copy the **.vcd** to a desktop machine and open it there.

SECTION 5 · RTL TO GATES

Synthesis Flow

Turn the Verilog you wrote into the actual logic gates a chip would be built from.

Yosys is driven by a script so the flow is repeatable: **read_verilog** → **hierarchy** → **proc** → **techmap** → **dfflibmap** → **abc** → **clean** → **write_verilog**.

RUN IT

```
$ igny run yosys -s yosys_commands.tcl
```

How to check the result

Yosys prints the cell count and area after dfflibmap and abc. Then open synth_example.v - it should contain only Nangate cell instances and wires, none of your original always/assign lines.

A MINIMAL DESIGN TO TRY IT ON

```
module top(a, b, clk, select, out);  
  input  a, b, clk, select; output out; reg out; wire y;  
  
  assign y = (select) ? b : a;    // mux  
  always @(posedge clk) out <= y; // → D flip-flop  
endmodule
```

SECTION 6 · SPICE AND PYTHON

Analog Flow

Transistor-level circuits, checked two ways: by equation and by simulation.

SETUP AND RUN

```
$ igny env create analog_experiments
$ igny workspace create --path analog_demo --env analog_experiments
$ cd analog_demo && igny tool install ngspice matplotlib

$ igny run script ./scripts/run_analysis.py # equation check
$ igny run ngspice -b -o build/spice.log spice/design.cir
$ cat ./build/spice.log
```

Batch mode or window mode

Use -b for DC operating-point runs - finishes and writes to the log, no window needed. Drop -b for a transient analysis you want to watch - the .control block opens a live window (needs a display; see section 8).

gm=1.00 mS gain=-6.90 V/V → PASS

A gap between the equation and SPICE results is information, not a bug - check the bias point first.

SECTION 7 · REPRODUCIBILITY

Sharing and Reproducing Work

The reason for all of this: someone else gets your exact toolchain, first try - any OS.

YOU: FREEZE AND PUBLISH

```
$ igny workspace lock  
$ git add . && git commit -m "lab setup"  
$ git push
```

THEM: CLONE AND RESTORE

```
$ git clone <repo> && cd <repo>  
$ igny workspace sync
```

igny workspace sync reads the lock and installs precisely those versions. If your results reproduce, it is because the toolchain genuinely is identical.

GOOD HABITS

- Commit crucible.toml and the lock file - they are the whole point.
- Do not commit build outputs - .vvp, .vcd, build/ regenerate in seconds.
- Re-run igny workspace lock whenever you install a new tool.

SECTION 8 · PLATFORM NOTES

Linux-Specific Notes

What's different from the Windows edition - distro by distro.

TOOLCHAIN INSTALL, BY DISTRO

DISTRO	TOOLCHAIN INSTALL	NOTES
Debian / Ubuntu	<code>sudo apt install build-essential</code>	Covers WSL2 (Ubuntu image).
Fedora / RHEL	<code>sudo dnf groupinstall "Dev Tools"</code>	No extra action needed.
Arch / Manjaro	<code>sudo pacman -S base-devel</code>	One-line group install.

WSL2 specifically

Keep your workspace inside the Linux filesystem (~/my-lab), not under /mnt/c/... - crossing the boundary is slow and can produce mismatched-path build errors. GTKWave needs WSLg or an X server. Everything else behaves identically to native Linux.

Coming from a Windows-built workspace

If a workspace's build directory was ever shared with a Windows checkout of the same path, stale artifacts with Windows-style paths can break a Linux build. Clean the build directory first.

SECTION 9 · WHEN THINGS BREAK

Troubleshooting

What the message means, and the one thing to try first.

WHAT YOU SEE	FIX
Tool not found / not recognised	igny tool install <tool>, then use igny run.
Not a workspace / no env found	cd back to the folder with crucible.toml.
g++: No such file or directory	Install build-essential - section 8.
iverilog: no top-level module	Add -s <testbench_module>.
GTKWave will not open / no display	ssh -X, enable WSLg, or copy the .vcd.
Permission denied installing a tool	Don't use sudo - igny installs under \$HOME.

Still stuck?

Copy the full command you ran and the complete output - not just the last line - and send it to the Ignytion team.

Every command, one page

```
# setup
igny init
igny env create <env>
igny workspace create --path <dir> --env <env>
igny tool install <tool>
igny workspace lock / sync

# digital
igny run iverilog -o sim.vvp -s tb_mod design.v tb.v
igny run vvp sim.vvp
igny run gtkwave tb.vcd

# synthesis
igny run yosys -s yosys_commands.tcl

# analog
igny run script ./scripts/run_analysis.py
igny run ngspice -b -o build/spice.log spice/design.cir
cat ./build/spice.log
```

Run from the workspace root · prefix every tool with igny run · read the first error, not the last.